

RENTipid Numbered Application Documentation

Complete Module Specification, Governance, Operations, and
Acceptance Reference

RENTipid

13 August 2026

- RENTipid Numbered Application Documentation
 - 1. Document control
 - 2. Application overview
 - 3. Governance, status, and acceptance
 - 4. Group A — Platform Foundation (FND)
 - 5. Group B — Identity and Account Management (IDN)
 - 6. Group C — Provider System (PRV)
 - 7. Group D — Listings and Marketplace (MKT)
 - 8. Group E — Rental Transaction Engine (TXN)
 - 9. Group F — Payments and Escrow (PAY)
 - 10. Group G — Trust and Protection (TRU)
 - 11. Group H — Communications and Autonomous Support (COM)
 - 12. Group I — Admin and Operations (ADM)
 - 13. Group J — SOC and Cybersecurity (SEC)
 - 14. Group K — Analytics and Mobile (ANA/MOB)
 - 15. Group L — Legal and Documentation (LEG/DOC)
 - 16. Group M — Release (REL)
 - 17. Supplemental module — Social Media, Promotion, and Feedback Intelligence
 - 18. Cross-module state and ownership rules
 - 19. Data architecture and records
 - 20. Interfaces, APIs, services, and integrations
 - 21. Security, privacy, legal, and financial controls
 - 22. Testing and acceptance strategy
 - 23. Deployment, operations, and recovery
 - 24. Current limitations and non-live boundaries
 - 25. Module dependency order
 - 26. Responsibility and change control
 - 27. Traceability summary
 - 28. Glossary
 - 29. Final application statement

RENTipid Numbered Application Documentation

1. Document control

1.1 Document identity

Field	Value
Document	RENTipid numbered application documentation and module specification
Edition	1.0
Date	13 August 2026 (Asia/Shanghai)
Repository	RENTipid
Branch inspected	feature/soc-phase4-threat-response
Commit inspected	e37e8e5514cf6bb24618723e66ebb8d36739799a
Module-status baseline	11 August 2026 register at 6f55296cdf1ff2bda3c550448fc307f264f1f397
Classification	Internal product, engineering, operations, security, compliance, and training reference
Core scope	49 registered modules in Groups A-M
Extension scope	1 supplemental module, SOC-01
Total documented modules	50

1.2 Purpose

This document is the numbered, systematic application reference for RENTipid. It explains what the application is, who uses it, how its principal journeys work, what each registered module owns, how modules interact, what evidence is required for promotion, and which limitations prevent a capability from being represented as live or complete.

This is a documentation consolidation, not a new release declaration. The presence of a route, model, test, integration adapter, infrastructure definition, or historical acceptance record does not by itself prove a currently deployed production capability.

1.3 Intended audiences

1. Product Owner and decision makers.
2. Product, design, and business analysts.
3. Renters and individual or business providers.

4. Admin, Finance Admin, Compliance Admin, SOC, privacy, and support personnel.
5. Developers, database engineers, platform operators, and maintainers.
6. Test, security, legal, audit, and release reviewers.

1.4 Source-authority order

If documents conflict, use this order:

1. Current executable code and schema establish what is implemented.
2. Accepted closure and freeze records establish what was formally accepted within an exact scope.
3. The Master Module Register establishes module IDs, promotion gates, and open evidence at its recorded baseline.
4. The Master Plan establishes required target scope, closure gates, and cross-module acceptance journeys.
5. Working registries establish detailed route, API, data, role, integration, security, test, and workflow inventories.
6. Historical plans describe intent and must not override current implementation or later accepted decisions.

1.5 Companion references

This numbered guide is the module-level entry point. Detailed inventories and manuals remain in:

1. Complete System Documentation — consolidated product, user, technical, security, operations, AI, insurance, privacy, profile, registry, inventory, and diagram volumes.
2. Final Documentation Index — canonical manual and evidence navigation.
3. Status Terminology Registry — controlled meaning of route, capability, operational, and evidence states.
4. Known Gap Register — registered implementation and evidence gaps.
5. Closure Register and Freeze Register — accepted closure boundaries.

2. Application overview

2.1 Product definition

RENTipid is a multi-role rental marketplace. It connects renters with individual and business providers and coordinates the controlled lifecycle of a rental: account creation, verification, discovery, listing, availability, booking, pricing, agreement, payment, handover, active rental, return, claims, disputes, review, settlement, support, administration, privacy, and security operations.

The product is not only a listing directory. Its central design concern is an evidence-backed transaction whose ownership, state changes, money movement, permissions, and exceptional outcomes can be reviewed and reconciled.

2.2 Principal actors

Actor	Principal responsibilities	Authority boundary
Guest	Browse public information, discover listings, read policy, register, and sign in	No private marketplace or operational mutation
Renter	Manage own profile; search, book, accept agreements, pay through authorized modes, inspect, return, claim, dispute, and review	Own records and permitted transaction actions only
Individual Provider	Complete onboarding/KYC; manage own listings, availability, fulfillment, evidence, earnings, and payout information	No publication, compliance, finance, or platform override unless explicitly granted
Business Provider	Manage business identity, KYB, authorized staff-facing workflows, listings, bookings, and approved promotion consent	Business scope does not imply administrative authority
Admin	Operate general marketplace, bookings, disputes, support, categories, UAT, and approved configuration surfaces	Finance, compliance, and SOC privileges remain separately controlled
Finance Admin	Review transactions, ledger, deposits, refunds, payouts, and reconciliation	Cannot activate prohibited live-money flows or bypass dual control
Compliance Admin	Review KYC/KYB, documents, listings, restricted items, and privacy/compliance evidence	Minimum necessary access; no general finance or security-response authority
SOC Analyst	Review security events and alerts, investigate cases, add evidence, and request response	Cannot self-approve a privileged response
SOC Supervisor	Review playbooks, approvals, grants, execution, and rollback	Must preserve approval scope, separation of duties, and emergency-freeze rules
Super Admin	Govern high-risk platform configuration, RBAC, feature flags, and emergency controls	Cannot erase legal, privacy, finance, audit, NO-GO, or dual-control obligations

2.3 Architecture summary

1. The user-facing application is a Next.js App Router application designed for Vercel hosting.
2. PostgreSQL is the authoritative relational datastore and Prisma is the principal schema/client layer.
3. Root Next.js API handlers coexist with an extracted Express API and worker under `apps/`; this is a partially split runtime, not proof that all Azure services are deployed.
4. The target backend direction uses Azure services, including compute, storage, secrets/identity, telemetry, search, and AI where authorized.
5. External capabilities use adapters so mock, sandbox, disabled, and real-provider modes can remain explicit.
6. High-risk actions are expected to enforce server-side authentication, RBAC, ownership, validation, audit, idempotency, and transaction controls.
7. PWA and Capacitor provide the mobile delivery direction while retaining the same authoritative backend behavior.

2.4 Core renter journey

1. Register and authenticate.

2. Complete profile, address, and verification requirements.
3. Search and inspect a listing.
4. Select an available rental period.
5. Create a booking and receive authoritative pricing.
6. Satisfy the authorized payment gate.
7. Accept a versioned rental agreement.
8. Record handover condition and acknowledgements.
9. Complete the active rental.
10. Record return condition and exceptional charges, if any.
11. Resolve claim, dispute, deposit, refund, or insurance branches.
12. Close the transaction and submit an eligible review.

2.5 Core provider journey

1. Register as an individual or business provider.
2. Complete provider profile and KYC/KYB.
3. Create a listing with media, price, deposit, location, availability, terms, and specifications.
4. Submit the listing for validation, compliance review, and publication.
5. Receive and manage eligible bookings.
6. Perform handover and return evidence workflows.
7. Address claims or disputes.
8. Review earnings, deductions, settlement, and payout evidence.
9. Use messaging, notifications, reviews, and approved promotion features within granted scope.

2.6 Exceptional journeys

1. **Damage:** booking → handover evidence → return evidence → claim → responses → determination → deduction/refund/insurance branch.
2. **Insurance:** eligible booking → quote/offer → selection → policy/coverage record → claim → partner response → settlement; current live partner activation remains a separately governed concern.
3. **Finance:** payment → gateway event → ledger/holding → fee/deduction → payout or refund → reconciliation.
4. **Support:** contextual help → deterministic answer or safe action → support case → human escalation when policy or risk requires it.
5. **Security:** suspicious/unauthorized action → block or event → detection/alert → incident case → approved response → verification or rollback → audit.

3. Governance, status, and acceptance

3.1 Mandatory promotion sequence

Every module is governed by the master sequence:

1. CODE COMPLETE
2. LOCAL FUNCTIONAL
3. LOCAL DATABASE MIGRATED
4. LOCAL REQUIRED DATA SEEDED/SYNCD
5. LOCAL ACCEPTANCE PASS

6. COMPLETED
7. CLOSED
8. FROZEN
9. PREVIEW MIGRATED
10. PREVIEW ACCEPTANCE PASS
11. PRODUCTION-READY

No gate may be inferred from a later-looking artifact. A gate is supported only by evidence for the whole stated module scope.

3.2 Current-status interpretation

1. `IN IMPLEMENTATION` means required behavior or evidence is missing.
2. `NOT STARTED` means the required capability was not found at the register baseline.
3. `LOCAL ACCEPTANCE PASS` means local evidence exists for the stated scope; it does not mean preview or production approval.
4. `CLOSED` / `FROZEN` applies only to the exact accepted scope and baseline.
5. A historical freeze remains protected but does not automatically satisfy the newer preview and production chain.
6. `COMPLETE_NO_GO_FROZEN` means a governance phase is complete while live activation remains prohibited.
7. External state is unverified unless deployment/provider evidence proves it.

3.3 Standard closure evidence

Each module must have:

1. **Code evidence:** no in-scope TODO, placeholder presented as real, dead route, mock masquerading as production, or duplicate authority.
2. **Database evidence:** schema, constraints, migrations, fresh-database proof, required data, and transaction boundaries.
3. **Functional evidence:** happy, negative, edge, permission, ownership, and cross-module paths.
4. **Security evidence:** authentication, authorization, IDOR protection, validation, audit, secret handling, and abuse controls.
5. **Acceptance evidence:** consolidated local pass, dispositioned defects, and archived results.
6. **Closure evidence:** completion decision, independent review, immutable baseline, freeze manifest, and controlled reopen procedure.

4. Group A — Platform Foundation (FND)

4.1 FND-01 — Architecture, configuration, PostgreSQL, and Prisma

Purpose. Provide the runtime, development, configuration, database, API, UI, and dependency foundation used by every other module.

Required scope. Next.js/React shell, TypeScript strictness, UI system, server/client boundaries, local setup, PostgreSQL connectivity, Prisma, API architecture, environment validation, feature flags, errors, logging, and health behavior.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. App Router, Prisma, an Express API, and worker exist, but configuration remains split and the production environment contract is incomplete and fragmented.

Acceptance boundary. A clean checkout must start using documented commands without undocumented repair. Configuration must fail closed, secrets must remain outside source, and runtime ownership between root APIs and extracted services must be explicit.

4.2 FND-02 — Migrations

Purpose. Make every schema change ordered, reviewable, repeatable, and safe across clean, existing, preview, and production databases.

Required scope. Migration history, additive/destructive-change review, constraints and indexes, rollback/recovery strategy, drift detection, fresh-database rehearsal, and migration status evidence.

Current registered state. IN IMPLEMENTATION; next permitted gate: LOCAL DATABASE MIGRATED. The baseline register records 36 applied migrations, with password-recovery and Insurance Slice 1 migrations validated but unapplied at that time.

Acceptance boundary. Prove empty database → migrations → required data → application startup → tests. Manual SQL is allowed only as a documented emergency or controlled recovery procedure.

4.3 FND-03 — Seeds and required data

Purpose. Supply deterministic data that the application requires to function, without mixing required data with disposable demos.

Required scope. Roles/permissions, settings, categories, policy/prohibited-item rules, workflow data, PSGC/address data, test fixtures, UAT data, idempotent execution, and a versioned required-data manifest.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Core, marketplace, prohibited-item, E2E, UAT, and PSGC paths exist, but required settings, roles, and workflow data are not unified in one deterministic manifest.

Acceptance boundary. Seed/sync commands must be repeatable and environment-safe, prove required counts/keys, and never silently write test identities into production.

4.4 FND-04 — Health, errors, and logging

Purpose. Report truthful readiness, return consistent failures, and create usable diagnostic evidence without leaking sensitive data.

Required scope. Next and extracted-service health/readiness, database dependency checks, structured errors, correlation IDs, application logs, sanitization, and operator troubleshooting.

Current registered state. LOCAL ACCEPTANCE PASS; next permitted gate: PREVIEW MIGRATED, held by the global preview barrier. Both Next and Express probes query PostgreSQL and fail closed; focused local tests and localhost checks passed for the accepted scope.

Acceptance boundary. A response must distinguish process liveness from dependency readiness. A failed database must never produce a healthy readiness result.

4.5 FND-05 — Audit trail

Purpose. Preserve who did what, to which object, when, through which path, with an outcome suitable for investigation and reconciliation.

Required scope. Authentication, RBAC, profile, KYC, listings, bookings, agreements, payments, refunds, payouts, claims, disputes, insurance, admin, security, privacy, configuration, and failure events.

Current registered state. LOCAL ACCEPTANCE PASS; next permitted gate: PREVIEW MIGRATED. Audit, authentication, API, payment, and SOC writers exist; whole-application sensitive-mutation coverage is not yet proven.

Acceptance boundary. Audit data must be append-oriented, access-controlled, sanitized, correlated, and protected from unauthorized change. Coverage must be proven by mutation category, not assumed from a shared helper.

5. Group B — Identity and Account Management (IDN)

5.1 IDN-01 — Registration, login, logout, and sessions

Purpose. Establish and maintain authenticated user identity and safe account access.

Required scope. Registration, credential validation and hashing, login/logout, sessions/JWT lifecycle, expiry, disabled/locked accounts, recovery, security events, and notification.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Core registration/login/logout exist and fail-closed account-state repairs are recorded, but the recovery schema was unapplied and SMTP/reset handlers required explicit authorization at the baseline.

Acceptance boundary. Recovery tokens must expire, be single-use, resist enumeration, and authorize only the intended credential mutation. Session state must reflect account disablement and privilege changes without optimistic access.

5.2 IDN-02 — MFA and session step-up

Purpose. Require stronger proof before privileged or high-risk actions.

Required scope. MFA enrollment/challenge/recovery, step-up policy, session assurance level, expiry, replay resistance, audit, and negative authorization tests.

Current registered state. LOCAL ACCEPTANCE PASS; next permitted gate: PREVIEW MIGRATED. Historically accepted under Security Phase 5C; new-standard preview evidence is not recorded for the isolated scope.

Acceptance boundary. Possession of a normal session must not substitute for step-up where policy requires it, and recovery cannot become a weaker bypass.

5.3 IDN-03 — Profile and account settings

Purpose. Let users safely manage their personal/business identity, contact data, image, password, preferences, and permitted account settings.

Required scope. Read/edit, field validation, ownership, photo lifecycle, sensitive-field policy, password change, account state, and admin-managed fields.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Personal/business profiles, photo, and password change exist; the register states that complete settings/status and consolidated journey evidence remain open. Later profile-program freeze records must be reconciled by exact scope before promotion.

Acceptance boundary. Server-side field allowlists, IDOR protection, encryption for designated fields, old-password/step-up checks, audit, and user/admin separation are mandatory.

5.4 IDN-04 — Global Address and PSGC

Purpose. Provide a normalized global address model with authoritative Philippine geographic data and secure ownership.

Required scope. Country, region, province, city/municipality, barangay, postal code, lines, normalization, validation, encrypted persistence, authority tokens, and transaction integrity.

Current registered state. CLOSED / FROZEN; no open blocker. Frozen at 6f55296cdf1ff2bda3c550448fc307f264f1f397 with a complete local-to-preview-to-production-readiness chain.

Acceptance boundary. Any modification requires controlled change and affected-scope revalidation; unrelated work must not reopen the module.

5.5 IDN-05 — Roles and permissions / RBAC

Purpose. Enforce least privilege across pages, APIs, services, data ownership, and administrative actions.

Required scope. Guest, Renter, Individual Provider, Business Provider, Admin, Finance Admin, Compliance Admin, SOC roles, Super Admin, permission vocabulary, route/API mapping, UI visibility, escalation protection, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Core and SOC permission systems exist; dual role systems and string-backed database roles require one authoritative matrix and complete route/API coverage proof.

Acceptance boundary. UI hiding is not authorization. Every protected mutation must enforce session, role/permission, ownership/scope, object state, and audit on the server.

6. Group C — Provider System (PRV)

6.1 PRV-01 — Individual provider onboarding

Purpose. Convert an eligible user into an activated individual provider through a controlled, reviewable journey.

Required scope. Registration choice, onboarding checklist/wizard, profile completeness, requirements, KYC, verification state, activation, rejection/resubmission, and dashboard entry.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Checklist, profile, KYC, and listing surfaces exist; focused end-to-end onboarding acceptance is missing.

Acceptance boundary. An incomplete or rejected provider must not publish or fulfill rentals. State transitions and reviewer actions require evidence.

6.2 PRV-02 — Business provider onboarding and KYB

Purpose. Establish a verified business identity and its authorized marketplace scope.

Required scope. Business registration, legal/profile data, representatives, documents, KYB state, ownership/control evidence, activation, listing access, and provider promotion consent.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Business registration/profile/marketing surfaces exist, but the business path is incomplete and real social integrations are absent.

Acceptance boundary. Business authority, user membership, consent, and document access must be scoped explicitly; a business profile cannot grant general admin rights.

6.3 PRV-03 — KYC/KYB document verification

Purpose. Securely collect, validate, review, expire, reject, and resubmit identity/business evidence.

Required scope. Upload validation, metadata, secure storage, malware/content controls, verification lifecycle, expiry where applicable, reviewer actions, minimum-necessary reads, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. UI, models, and compliance surfaces exist; relevant Vercel routes return 410 and depend on an Azure backend whose deployed runtime was unproven.

Acceptance boundary. No document may be public or exposed by guessable ownership. Adapter/storage health and extracted-route ownership must be proven in the target environment.

7. Group D — Listings and Marketplace (MKT)

7.1 MKT-01 — Categories and marketplace required data

Purpose. Provide the controlled taxonomy and minimum reference data needed for listing and discovery.

Required scope. Categories, specifications, policy mappings, seeded sample/acceptance inventory, stable identifiers, ordering, and administrative governance.

Current registered state. LOCAL REQUIRED DATA SEEDDED/SYNCD; next permitted gate: LOCAL ACCEPTANCE PASS. The register records accepted local evidence for 15 categories and 45 listings; whole marketplace workflow acceptance remains open.

Acceptance boundary. Required data must be deterministic and compatible with listing, filter, prohibited-item, analytics, and migration behavior.

7.2 MKT-02 — Listing create, edit, lifecycle, and publication

Purpose. Let authorized providers create and manage accurate rentable inventory through a controlled publication lifecycle.

Required scope. Category, title, description, media, price, deposit, location, availability, terms, specifications, draft, validation, compliance, approval, publish, pause, unpublish, archive, edit ownership, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Create, submit, review, and display paths exist; provider-detail editing is represented as disabled and lifecycle parity across Next/Azure paths is unproven.

Acceptance boundary. The authoritative lifecycle is DRAFT → VALIDATION → APPROVED → PUBLISHED → PAUSED → UNPUBLISHED/ARCHIVED, with only permitted transitions and re-review where material changes demand it.

7.3 MKT-03 — Listing media and storage

Purpose. Store, validate, order, retrieve, and retire listing photos/documents safely.

Required scope. Upload authorization, file type/size/content checks, metadata, object ownership, private/public boundaries, image order, deletion, provider abstraction, and recovery.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Local/Azure and document/photo paths exist; S3, R2, and Supabase adapters contain not-implemented behavior, and Azure runtime availability was unproven.

Acceptance boundary. Unsupported adapters must fail explicitly. No local path, credential, private document, or cross-tenant object may be exposed.

7.4 MKT-04 — Prohibited/restricted listing compliance

Purpose. Prevent disallowed inventory from becoming available and preserve a reviewable compliance decision.

Required scope. Prohibited rules, restricted categories, automated checks, policy engine, bounded AI assistance, blocking, reviewer/appeal flow, enforcement, audit, and dedicated negative tests.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Models, seed, service, admin UI, and tests exist, but historical freeze claims conflict with failed later closeout evidence and the enforcement UI contains a placeholder.

Acceptance boundary. This module requires independent acceptance. AI may assist classification but cannot silently override deterministic policy or required human review.

7.5 MKT-05 — Search, filters, and discovery

Purpose. Help users find eligible, visible, relevant listings without leaking private or disallowed inventory.

Required scope. Keyword, category, location, price, availability, filters, sorting, pagination, details, recommendations, empty/error states, and safe fallback behavior.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Browse/detail/category filtering and an Azure search service exist; focused acceptance, authorization boundaries, and fallback behavior are not proven.

Acceptance boundary. Search results must respect publication/compliance/availability state. Index freshness, query validation, pagination stability, and fallback parity require evidence.

7.6 MKT-06 — Availability and locking

Purpose. Prevent double booking and ensure availability remains authoritative throughout transaction changes.

Required scope. Calendar/range checks, overlap rules, holds, booking conflicts, expiry/release, provider updates, concurrency, transaction boundaries, and timezone handling.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Availability checks and booking conflict logic exist; full lifecycle concurrency/locking proof is not accepted.

Acceptance boundary. Concurrent requests must produce at most one valid reservation for the same exclusive interval, with rollback and expired-hold recovery.

8. Group E — Rental Transaction Engine (TXN)

8.1 TXN-01 — Booking and pricing

Purpose. Create a bookable transaction with authoritative dates, availability, amount, deposit, fee, discount, insurance, and tax components.

Required scope. Requested/accepted/payment-pending/confirmed/active/completed states; declined/cancelled/expired/disputed/refunded exceptions; price units, rounding, snapshots, and booking history.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Booking creation, price units, history, and dashboards exist; the complete renter/provider journey and one authoritative pricing/fee contract are missing.

Acceptance boundary. Price must be calculated server-side from versioned inputs and persisted as a transaction snapshot. Availability, money, and state changes require atomicity where necessary.

8.2 TXN-02 — Rental agreement and acceptance

Purpose. Bind the transaction parties to a stable, reviewable set of rental terms.

Required scope. Renter/provider/item, dates, amounts, deposits, cancellation, damage responsibilities, relevant policy versions, generated agreement, signatures/acceptance, timestamp, and immutable version.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Agreement APIs, pages, and model exist; agreement and legal-policy version recording remain incomplete.

Acceptance boundary. Later listing or policy edits cannot retroactively change the accepted agreement. Each party's acceptance and authority must be recorded.

8.3 TXN-03 — Handover, active rental, and return

Purpose. Establish item condition and custody at both ends of the rental.

Required scope. Checkout/check-in inspections, photos, checklist, timestamp, acknowledgements, active state, missing items, damage, lateness, extra-charge proposal, and final closure.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Inspection, turnover, renter confirmation, claim, and return pages exist; complete state-machine, recovery, and end-to-end evidence are missing.

Acceptance boundary. Evidence must be booking-linked, time-ordered, ownership-protected, and immutable or versioned after acknowledgement.

8.4 TXN-04 — Cancellation and expiration

Purpose. End or release transactions consistently when parties cancel, payment is not completed, or a hold/request expires.

Required scope. Eligibility, actor/reason, policy calculation, notification, availability release, refund branch, worker sweep, retries, idempotency, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Status flows and a worker sweeper exist; worker scheduling/deployment and focused expiration tests are unproven.

Acceptance boundary. Expiration must be safe under retries and concurrency and must never release availability belonging to a later valid state.

9. Group F — Payments and Escrow (PAY)

9.1 PAY-01 — Payment checkout and gateway transactions

Purpose. Initiate and record a payment through an explicitly authorized gateway mode.

Required scope. Checkout/payment intent, success/failure/cancellation, amount/currency integrity, transaction reference, adapter mode, idempotency, audit, and user return flow.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Mock and PayMongo adapters, checkout, and transaction records exist; sandbox/live boundaries need reconciliation and live activation remains prohibited.

Acceptance boundary. Client-reported success is never authoritative. Only verified gateway evidence and reconciled internal state may confirm money movement.

9.2 PAY-02 — Webhooks and financial idempotency

Purpose. Accept asynchronous provider events without forgery, duplication, reordering corruption, or double financial effect.

Required scope. Signature verification, raw-payload requirements, event log, deduplication, idempotency keys, event-state mapping, replay/retry, correlation, and reconciliation.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Signature, event log, reconciliation, and checkout-idempotency controls exist; a complete accepted callback matrix is missing.

Acceptance boundary. Invalid signatures fail closed; duplicates produce no duplicate ledger or state effect; out-of-order events cannot regress an authoritative state.

9.3 PAY-03 — Escrow/holding and ledger

Purpose. Represent renter funds, held amounts, fees, deductions, refunds, and provider payables using balanced, explainable accounting records.

Required scope. Payment, hold, release, fee, deposit, damage/dispute/insurance adjustments, provider payable, ledger invariants, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Payment, gateway, ledger, and deposit records exist; legal escrow semantics and invariant proof are incomplete.

Acceptance boundary. “Escrow” must not be claimed beyond the legal/provider arrangement. Every balance-changing operation must be idempotent, attributable, and reconcilable.

9.4 PAY-04 — Refunds

Purpose. Return full or partial eligible funds with policy, gateway, ledger, and duplicate controls.

Required scope. User request, admin review, full/partial/cancellation refund, gateway execution, failure/retry, ledger entries, status, reconciliation, and notification.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Request/admin surfaces exist; the PayMongo refund method is a success-returning placeholder and live refunds are manual.

Acceptance boundary. A placeholder response must never mark real money refunded. Internal status must distinguish requested, approved, submitted, succeeded, failed, and manually settled.

9.5 PAY-05 — Provider payouts

Purpose. Calculate and settle the provider's payable amount after fees, deductions, refunds, and holds.

Required scope. Payable calculation, request/batch, beneficiary readiness, execution, settlement, failure, retry, statements, reconciliation, and separation of duties.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Payout/batch records and admin surfaces exist; real payout execution is a manual placeholder.

Acceptance boundary. A payout may be represented as paid only with verified provider/bank evidence and matched ledger effect.

9.6 PAY-06 — Financial reconciliation

Purpose. Prove that RENTipid records agree with gateway and settlement evidence and make discrepancies actionable.

Required scope. Payment, refund, fee, deposit, holding, payout, settlement, variance classification, exception queue, correction approval, export, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Reconciliation screens/logs exist; end-to-end equality and discrepancy handling across all financial paths are not proven.

Acceptance boundary. No unexplained difference may remain at closure. Corrections must preserve original evidence rather than rewriting history.

10. Group G — Trust and Protection (TRU)

10.1 TRU-01 — Insurance

Purpose. Offer provider-neutral insurance capabilities without implying a live insurer or coverage before activation.

Required scope. Eligibility, quote/offer, premium, checkout selection, policy/coverage record, issuance integration, cancellation/refund, claim/evidence, insurer response, settlement, finance reconciliation, audit, and provider adapter governance.

Current registered state. Technical Foundation Slice 1 is CLOSED / FROZEN; the full module is IN IMPLEMENTATION. The slice is frozen at 2ff068991950de64e3bf0931ed76a5650217dbe2; booking/auth integration, routes, lifecycle, finance, and real partner activation remain open.

Acceptance boundary. Engineering closure and live insurer activation are separate decisions. No mock quote, adapter, record, or UI may be presented as active coverage.

10.2 TRU-02 — Damage claims and evidence

Purpose. Resolve alleged rental damage using transaction-linked evidence and controlled financial outcomes.

Required scope. Claim creation, images/evidence, agreement link, estimated loss, renter/provider responses, assessment, deposit deduction, insurance linkage, partial outcome, determination, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Claims, photos, response, and admin resolution records exist; determination-to-ledger acceptance is missing.

Acceptance boundary. A claim decision must not create an unexplained financial effect. Evidence access, deadlines, conflicts, and appeal/dispute transitions require explicit rules.

10.3 TRU-03 — Disputes

Purpose. Provide a fair case workflow when parties contest a transaction, claim, cancellation, or financial result.

Required scope. Initiation, classification, transaction hold/freeze where authorized, evidence, communications, assignment, decision, adjustment, closure, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. A dispute model and admin resolution surfaces exist; renter/provider case workflow and dedicated tests are incomplete.

Acceptance boundary. The decision maker, evidence visibility, state transitions, financial authority, and appeal/reopen policy must be unambiguous.

10.4 TRU-04 — Reviews and reputation

Purpose. Record eligible transaction-based feedback and produce resistant-to-manipulation reputation signals.

Required scope. Renter/provider review, eligibility, one review per transaction/side, rating calculation, moderation/reporting, aggregate score, timing, and anti-abuse rules.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. A review model and read paths exist; dedicated mutation workflow and focused tests were not found.

Acceptance boundary. Only eligible completed transactions may affect reputation; moderation must not silently rewrite the original review record.

11. Group H — Communications and Autonomous Support (COM)

11.1 COM-01 — Direct messaging

Purpose. Enable safe renter-provider communication in the context of an authorized booking or marketplace relationship.

Required scope. Conversations, participants, booking link, messages, timestamps, read state, system messages, abuse controls, retention, and authorized attachments.

Current registered state. NOT STARTED; next permitted gate: CODE COMPLETE. No conversation/message model, API, or UI was found at the register baseline.

Acceptance boundary. Conversation membership must be server-enforced. Attachments and contact exchange require policy, storage, and abuse review.

11.2 COM-02 — Notifications

Purpose. Deliver deduplicated, actionable event information through supported channels.

Required scope. In-app inbox, read/unread state, booking, payment, claim, payout, verification, security and account events, preferences, email where configured, retries, and duplicate prevention.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Notification persistence is used by booking creation; no complete inbox/API/read-state workflow or focused tests exist.

Acceptance boundary. Notifications are not the authoritative transaction state. Sensitive content must be minimized for external channels.

11.3 COM-03 — AI Help Center

Purpose. Provide contextual, transaction-aware assistance and bounded automated resolution while escalating unsafe, legal, financial, or ambiguous decisions.

Required scope. Knowledge retrieval, renter/provider/account/booking/payment/claim/policy support, tool gateway, policy engine, prompt/injection controls, case handoff, audit, privacy, resilience, and provider modes.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. UI, logs, prompt controls, and a mock command layer exist; the Vercel endpoint returns 410, Azure tool dispatch is unimplemented, and answer/tool paths are mock-only at the baseline.

Acceptance boundary. AI cannot approve money movement, KYC, publication, insurance coverage, privileged SOC response, or other prohibited decisions. Tool actions require deterministic authorization and auditable outcomes.

11.4 COM-04 — Support and transactional communications

Purpose. Capture user support, feedback, issue, and transactional communication outside direct renter-provider messaging.

Required scope. Ticket/feedback creation, categorization, ownership, case state, responses, escalation, transactional templates, email delivery, privacy handling, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Ticket/feedback models and admin reads exist; user forms are not wired to mutations and email delivery is not accepted.

Acceptance boundary. Support content is untrusted and may contain sensitive data; it requires validation, least-privilege access, retention, redaction, and safe notification rules.

12. Group I — Admin and Operations (ADM)

12.1 ADM-01 — Admin operations

Purpose. Give authorized operators consolidated marketplace visibility and controlled actions.

Required scope. Users, providers, categories, listings, bookings, claims, disputes, support, UAT/beta, reports, ownership assignment, state changes, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Broad dashboards and operational reads/actions exist; complete role journeys, ownership, audit, and negative acceptance are missing.

Acceptance boundary. A dashboard being visible does not authorize every action it links to. Each mutation must enforce the specialist role and object-state rules.

12.2 ADM-02 — Finance Admin

Purpose. Operate guarded money workflows and financial evidence with separation of duties.

Required scope. Payments, ledger, deposits/holding, refunds, payouts, batches, settlements, reconciliation, exceptions, exports, dual control, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Finance dashboards and manual controls exist; automated real-money operations and complete reconciliation acceptance are unavailable.

Acceptance boundary. Finance UI cannot override a provider result, NO-GO decision, or immutable ledger history.

12.3 ADM-03 — Compliance Admin

Purpose. Operate identity, business, listing, prohibited-item, privacy, and policy compliance workflows.

Required scope. KYC/KYB, verification documents, listing review, prohibited/restricted items, privacy records, policy violations, decision reasons, evidence, and audit.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE.

KYC/listing/privacy/prohibited-item surfaces exist; placeholder enforcement and conflicting prohibited-item evidence remain.

Acceptance boundary. Reviewer access follows minimum necessity; decisions require reason, actor, time, state precondition, and affected-subject notification where required.

12.4 ADM-04 — Super Admin and system controls

Purpose. Govern the highest-risk application controls without turning super-admin access into an audit or policy bypass.

Required scope. RBAC, settings, feature flags, payment/launch controls, emergency freeze, security configuration, privileged audit access, initialization, and break-glass governance.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Settings, launch, payment, and emergency surfaces exist; initialization is database-dependent and whole-scope authorization acceptance is missing.

Acceptance boundary. High-risk changes require step-up, reason, audit, and where applicable a second approver. Super Admin cannot override explicit Owner-reserved activation decisions.

13. Group J — SOC and Cybersecurity (SEC)

13.1 SEC-01 — Security/SOC

Purpose. Detect, investigate, contain, recover from, and learn from security events using reversible, authorized operations.

Required scope. Security event ingestion, normalization, lifecycle separation, deduplication, detection rules, alerts, behavioral risk, geolocation/threat map, incident cases, evidence, playbooks, approval grants, response, rollback, emergency freeze, simulations, maintenance, reporting, retention, and recovery.

Current registered state. LOCAL ACCEPTANCE PASS; next permitted gate: PREVIEW MIGRATED. Extensive historical Gate 4F-4J, Level 5, behavioral-intelligence, and threat-map slices are preserved; the new-standard preview chain is not recorded.

Acceptance boundary. Analysts cannot self-approve. A response must be within a valid grant, idempotent, concurrency-safe, auditable, reversible where promised, and blocked by emergency freeze where required.

13.2 SEC-02 — Privacy and consent v1

Purpose. Make personal-data collection, use, consent, subject rights, retention, deletion, processor handling, and security evidence governable.

Required scope. Public notices, policy versions, cookies/consent, correction/export/deletion requests, ownership, identity verification, retention, processors, cross-border records, encryption, audit, exceptions, and privacy incident handling.

Current registered state. LOCAL ACCEPTANCE PASS; next permitted gate: PREVIEW MIGRATED. Accepted local closure evidence exists; the recorded closure had no deployment, DPO registration and approved deferrals remain, and automated retention/deletion is not a live production assumption.

Acceptance boundary. Consent must be specific and versioned where required. Data-subject actions require identity/ownership proof and must not delete records retained for legitimate legal, fraud, security, or finance obligations without governed handling.

14. Group K — Analytics and Mobile (ANA/MOB)

14.1 ANA-01 — Analytics and KPIs

Purpose. Provide explainable product, marketplace, finance, support, trust, and security metrics from authoritative data.

Required scope. Renter/provider/listing metrics, bookings, conversion, GMV, revenue, fees, cancellations, refunds, claims, insurance, support, fraud/security, operational KPIs, filters, exports, and query reconciliation.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Multiple dashboards/models exist; mobile/events contain mock data and marketplace/finance KPI definitions lack accepted reconciliation.

Acceptance boundary. Every KPI needs definition, source, time basis, inclusion/exclusion rules, currency/rounding behavior, privacy classification, and a reconciled authoritative query.

14.2 MOB-01 — PWA

Purpose. Make the responsive web application installable and resilient within explicitly supported offline boundaries.

Required scope. Manifest, icons, service worker, installability, responsive layout/navigation, cache policy, update behavior, offline/error experience, and testing of principal user journeys.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. A manifest and brand icons exist; no service worker/offline behavior exists and placeholder 68-byte icons are also present.

Acceptance boundary. Sensitive or transaction-changing responses must not be unsafely cached. Offline UI cannot present a write as confirmed before server acceptance.

14.3 MOB-02 — Capacitor/mobile

Purpose. Package the authoritative web experience for native distribution without forking business logic.

Required scope. Capacitor configuration, platform projects, secure transport, deep links, storage/session handling, permissions, device testing, signing, store metadata, and release process.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Capacitor configuration and responsive shell exist; native platform projects/tests are absent and cleartext/mixed-content settings require hardening.

Acceptance boundary. Store readiness and publication are separate. Native configuration must not weaken web authentication, network security, privacy, or update controls.

15. Group L — Legal and Documentation (LEG/DOC)

15.1 LEG-01 — Legal and policy

Purpose. Integrate applicable terms, privacy, rental, cancellation, payment, insurance, prohibited-item, dispute, and data-processing obligations into user workflows.

Required scope. Published policy content, versioning, effective date, acceptance record, relevant declarations, withdrawal/change handling, and transaction-specific policy snapshots.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Terms, privacy, cancellation pages, and readiness UI exist; registration acceptance is not version-recorded and insurance/payment/rental consents are incomplete.

Acceptance boundary. Merely displaying a policy is not proof of agreement. Required acceptance must record user, version, purpose, timestamp, and transaction/context.

15.2 DOC-01 — Manuals and interface documentation

Purpose. Maintain accurate user, operator, technical, developer, deployment, and interface knowledge tied to current implementation and evidence.

Required scope. Guest/renter/provider manuals; Admin/Finance/Compliance/Super Admin procedures; architecture, database, API, environment, integration, security, monitoring, backup/recovery, repository/setup, migration, testing, CI/CD, change management, and screen behavior.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. A large historical documentation set exists, but the master register identifies reconciliation with current runtime as incomplete.

Acceptance boundary. Documentation status and application status are separate. Screenshots, counts, routes, provider states, and release instructions must be dated and evidence-linked.

16. Group M — Release (REL)

16.1 REL-01 — Global acceptance, LOCAL-RC1, closure, and deployment preparation

Purpose. Convert individually accepted modules into one controlled application release candidate and an evidence-backed deployment package.

Required scope. Clean build, production build, environment validation, migrations, required-data package, full journeys, security, financial reconciliation, documentation, no P0/P1 defects, monitoring, backup, rollback, version, tag, manifest, and change freeze.

Current registered state. IN IMPLEMENTATION; next permitted gate: CODE COMPLETE. Address is independently ready, but multiple modules remain below code complete and there is no global local acceptance or LOCAL-RC1.

Acceptance boundary. The required consolidated journeys are renter, provider, damage, insurance, finance, admin, and security. After LOCAL-RC1, only release-blocking corrections are allowed until closure.

17. Supplemental module — Social Media, Promotion, and Feedback Intelligence

17.1 SOC-01 — Social Media, Promotion, and Feedback Intelligence

Purpose. Support controlled promotion content, provider connections, scheduled publishing, analytics/attribution, feedback intelligence, and case handoff while keeping AI and external-provider behavior bounded.

Required scope. Social account registry, provider capability/health model, content studio, draft/version workflow, human review and approval, deterministic scheduler, publishing idempotency, metrics, attribution, provider events, feedback, RBAC, audit, secure credentials, media validation, and Unified AI integration.

Governed workflow. AI DRAFT → HUMAN REVIEW → AUTHORIZATION CHECK → HUMAN APPROVAL → DETERMINISTIC SCHEDULER → PROVIDER ADAPTER → PROVIDER RESULT → AUDIT.

Recorded state. PASS / FROZEN under feature/soc-phase4-threat-response, supported by the social architecture and Phase 12 acceptance records. The Mock adapter is the acceptance provider. Meta is scaffolded/partner-ready; TikTok, Google, WhatsApp, and Viber real credentials are not configured.

Acceptance boundary. Frozen engineering scope does not mean every real provider is active. AI cannot approve, schedule, or publish. Core logic must use the provider adapter and capability model, credential references must not expose secrets, and feedback must hand off to existing case-management boundaries.

Primary evidence. Social architecture, RBAC, provider adapters, content workflows, security/audit, testing/acceptance, and production operations.

18. Cross-module state and ownership rules

18.1 Authoritative state principles

1. The server and database are authoritative for identity, permission, ownership, transaction, finance, and security state.
2. UI labels are projections; they cannot create authority or prove completion.
3. Every mutation must validate the current state before applying a permitted transition.
4. Retried requests need idempotency where duplication would create state, money, notification, publication, or response effects.
5. Concurrent requests need transaction, version, unique-constraint, or lock protection appropriate to the invariant.
6. Historical evidence must be retained when a correction occurs; do not rewrite audit or ledger history to hide an error.

18.2 Principal state-machine families

Family	Representative progression	Exception handling
Provider verification	Draft/incomplete → submitted → under review → verified/activated	Rejected, resubmission, expired, suspended
Listing	Draft → validation → approved → published	Paused, rejected, unpublished, archived
Booking	Requested → accepted → payment pending → confirmed → active → completed	Declined, cancelled, expired, disputed, refunded
Agreement	Generated → presented → accepted by required parties → effective	Superseded before acceptance; immutable after accepted version
Inspection	Draft evidence → acknowledged → finalized	Disagreement creates claim/dispute branch
Payment	Initiated → provider pending → verified success → reconciled	Failed, cancelled, expired, disputed, refunded
Refund	Requested → reviewed → approved → submitted → succeeded → reconciled	Rejected, failed, retry/manual settlement
Payout	Calculated → approved → batched/submitted → paid → reconciled	Held, failed, retry, exception
Claim/dispute	Open → evidence/response → review → determination → financial resolution → closed	Escalation, appeal/reopen only by policy
SOC response	Requested → approved/granted → executing → succeeded/failed → verified	Freeze, rollback, divergence, recovery
Social post	Draft → submitted for review → approved → scheduled → published	Rejected, cancelled, provider failure, retry

18.3 Ownership rules

1. Users may access their own profile and subject-rights requests, subject to policy.
2. Providers may mutate only their owned or business-authorized inventory and fulfillment records.
3. Renters and providers may act on a booking only as a party and only in permitted states.
4. Specialists receive task-specific access: finance to finance, compliance to verification/policy, and SOC to security.
5. Cross-role views must minimize personal and financial data.
6. Super Admin does not eliminate separation of duties or reserved Owner decisions.

19. Data architecture and records

19.1 Data domains

1. **Identity:** users, sessions, accounts, MFA/recovery, profiles, addresses, roles, and consent.
2. **Provider/verification:** business profiles, provider state, verification documents, KYC/KYB decisions.
3. **Catalog:** categories, listings, specifications, photos/documents, locations, prohibited rules, and availability.
4. **Rental:** bookings, pricing snapshots, agreements, turnover, inspections, cancellations, claims, disputes, and reviews.
5. **Finance:** checkout, gateway transactions/events, ledger, deposits/holds, refunds, payouts, batches, and reconciliation.
6. **Trust:** insurance products/offers/policies/claims, evidence, determinations, and partner results.
7. **Communication:** notifications, support/feedback/issues, AI logs/cases, and eventually direct messages.
8. **Administration/release:** settings, UAT, beta, launch, feature flags, and release evidence.
9. **Security/privacy:** audit, authentication/API/payment/security events, rules, alerts, cases, evidence, approvals, responses, risk, geolocation, privacy requests, retention, and processors.
10. **Marketing/social:** campaigns, accounts, posts/versions, provider events, metrics, attribution, and opt-ins.

19.2 Data-control requirements

1. Schema and migration history must agree; drift requires controlled reconciliation.
2. Foreign keys, unique constraints, indexes, and delete policies must reflect domain ownership and retention.
3. Sensitive fields require classification, encryption where designated, access control, and sanitized audit/log output.
4. Finance uses immutable or append-oriented records with idempotent effects and reconciliation identifiers.
5. Evidence and accepted agreement/policy versions must remain stable after the relevant decision.
6. Test databases and seeds must be guarded from production connection strings.
7. Backup, restore, and recovery tests must validate usable data, not only archive creation.

20. Interfaces, APIs, services, and integrations

20.1 Interface contract

Every important screen should document:

1. Purpose and intended actor.
2. Required authentication and permission.

3. Fields, buttons, and validation.
4. Data source and authoritative state.
5. Happy, empty, loading, denied, invalid, conflict, and dependency-failure behavior.
6. State transitions and side effects.
7. Audit, notification, accessibility, responsive, and recovery behavior.

20.2 API/service contract

1. Define route ownership: root Next.js, extracted API, worker, or external provider callback.
2. Authenticate and authorize on the server.
3. Validate path, query, headers, and body using bounded schemas.
4. Enforce object ownership and current state.
5. Use transactions/idempotency/concurrency controls for critical effects.
6. Return consistent status/error shapes without leaking secrets or internals.
7. Correlate request, provider event, audit, and domain records.
8. Document retry, timeout, circuit-breaker, and recovery behavior.

20.3 External integration truth

Integration family	Intended use	Documentation rule
PayMongo/payment gateway	Checkout, asynchronous events, refund/payout-related evidence where supported	Sandbox/live must be explicit; live payment activation remains prohibited by current governance
Email/SMTP	Recovery and transactional communication	A configured variable or template is not delivery proof
Storage/Azure Blob	KYC and listing/claim media	Adapter existence is not deployed storage proof; private/public boundaries are mandatory
Azure AI Search/OpenAI	Search and authorized AI assistance	Disabled/mock/provider modes must be visible; tool actions remain deterministic
Insurance partner	Quote, policy, claim, and settlement	Technical foundation is not active coverage or insurer approval
KYC provider	Automated verification assistance	Human/legal policy and fallback remain explicit; provider activation requires evidence
Social providers	Promotion publishing, metrics, and feedback	Mock acceptance does not mean Meta/TikTok/Google/WhatsApp/Viber are configured
Geolocation/threat intelligence	Privacy-safe SOC enrichment	Provider mode, retention, IP handling, and failure behavior must be explicit
Vercel/Azure	Frontend and extracted backend/worker target	Local IaC/configuration is not provisioning, deployment, DNS, or traffic proof

21. Security, privacy, legal, and financial controls

21.1 Baseline security controls

1. Fail-closed authentication and account-state validation.
2. Central RBAC plus object ownership and business scope.
3. MFA/session step-up for privileged actions.
4. Input/schema validation and output minimization.
5. Upload type, size, path, ownership, malware/content, and storage controls.
6. Rate limiting and abuse protection on exposed/high-risk paths.
7. Secret separation, rotation, and managed-identity/key-vault direction.
8. Encryption and authenticated integrity for designated sensitive fields.
9. Sanitized, correlated, access-controlled audit/security logging.
10. Dependency/supply-chain, environment, database-safety, backup, recovery, and incident controls.

21.2 Privacy controls

1. Purpose and lawful-use documentation for collected data.
2. Versioned notices/consent where required.
3. Data minimization by user and operator role.
4. Identity/ownership verification for correction, export, and deletion requests.
5. Retention and deletion policy that accounts for legal, finance, fraud, dispute, and security obligations.
6. Processor and cross-border records.
7. Privacy incident escalation and evidence.
8. No inference that automated deletion is active when the accepted process is manual/deferred.

21.3 Financial controls

1. Amount and currency are server-authoritative.
2. Gateway signatures and raw callback rules are enforced.
3. Every external event and internal effect is idempotent.
4. Ledger effects balance and link to the initiating domain/provider evidence.
5. Refund and payout state distinguishes approval, submission, external success, and reconciliation.
6. Separation of duties applies to high-risk changes and exception correction.
7. COMPLETE_NO_GO_FROZEN and explicit activation prohibitions override readiness UI or code presence.

22. Testing and acceptance strategy

22.1 Test layers

1. Static analysis: formatting/lint and TypeScript.
2. Unit tests: calculations, policies, validation, mapping, permissions, and state predicates.
3. Service/integration tests: database transactions, ownership, idempotency, concurrency, audit, and adapter behavior.
4. API tests: authentication, RBAC, validation, status mapping, conflicts, and failures.
5. Browser journey tests: renter, provider, admin, finance, compliance, support, social, and SOC surfaces.
6. Fresh-database tests: migration and required-data replay.

7. Security tests: IDOR, escalation, injection, upload, webhook, replay, rate limit, secret leakage, and privileged workflow.
8. Operational tests: health, deployment rehearsal, rollback, backup/restore, worker recovery, monitoring, and dependency outage.

22.2 Required global acceptance journeys

1. **Renter:** register → profile → search → listing → book → authorized pay mode → agreement → rental → return → review.
2. **Provider:** register → KYC/KYB → provider profile → listing → publish → booking → handover → return → earnings → payout evidence.
3. **Damage:** booking → handover → damage evidence → claim → decision → deduction/refund.
4. **Insurance:** eligible booking → quote → selection → policy → claim → settlement, only within authorized provider mode.
5. **Finance:** payment → ledger/hold → fee → payout/refund → gateway reconciliation.
6. **Admin:** user → listing → booking → compliance → payment → claim → security → audit.
7. **Security:** unauthorized action → block/event → SOC → approved countermeasure → audit/rollback.

22.3 Defect and evidence rules

1. No open P0 or P1 defect is allowed for release candidate creation.
2. P2 defects must be fixed or formally dispositioned with owner, risk, and target.
3. Test-file presence is not pass evidence; record command, environment, database target, result, date, and artifacts.
4. Historical passes remain historical when later changes touch the accepted scope.
5. External integration acceptance must include real environment/provider evidence when claiming real operation.

23. Deployment, operations, and recovery

23.1 Local-to-release path

1. Establish a clean, documented local environment.
2. Validate configuration without revealing values.
3. Apply migrations and required data to an approved isolated database.
4. Run focused module and cross-module acceptance.
5. Create RENTipid LOCAL-RC1 only when every required local condition passes.
6. Freeze feature work; permit release-blocking fixes only.
7. Rehearse preview migration, acceptance, rollback, and monitoring.
8. Produce a versioned deployment package, manifest, checksums, migration list, dependency lock, evidence index, and recovery procedure.
9. Obtain separate authorization for production mutation, external-provider activation, DNS/traffic changes, or live money.

23.2 Operational readiness

1. Truthful liveness/readiness and dependency checks.
2. Structured logs, correlation, metrics, alert ownership, and escalation.

3. Database backups, verified restores, point-in-time objectives, and recovery runbooks.
4. Worker scheduling, leases, bounded retry/replay, dead-letter or failure recording, and checkpoint recovery.
5. Provider outage, rate limit, expired credential, invalid webhook, and degraded-mode procedures.
6. Emergency freeze and controlled unfreeze.
7. Rollback that accounts for code, schema, data compatibility, configuration, and external effects.

24. Current limitations and non-live boundaries

At the module-register baseline, the following statements are mandatory:

1. The complete application is not globally closed, frozen, preview-accepted, or production-ready.
2. Only the Address module has the accepted complete local-to-preview-to-production-readiness chain under the register baseline.
3. FND-04, FND-05, IDN-02, SEC-01, and SEC-02 have local acceptance but await the newer preview chain.
4. Direct messaging is not started.
5. Notification inbox/read-state and user support mutations are incomplete.
6. Payment refund and payout execution include manual or placeholder boundaries; live payment activation remains prohibited.
7. Legal escrow meaning and complete financial reconciliation are not proven.
8. Full insurance is not active; only the stated technical foundation slice is frozen.
9. AI Help/tool dispatch and Digital Human provider behavior must not be represented as live where routes are disabled or modes are mock/simulated.
10. Real social-provider credentials/activation are not implied by SOC-01 engineering acceptance.
11. PWA offline behavior and native Capacitor projects/tests are incomplete.
12. Local Azure/Vercel/IaC artifacts do not prove resources, deployment, DNS, traffic, production database migration, monitoring, or provider activation.
13. The working tree was dirty during documentation work; current uncommitted changes are not automatically part of any accepted baseline.

25. Module dependency order

1. **Foundation first:** FND-01 through FND-05, IDN-01 through IDN-05.
2. **Supply side:** PRV-01 through PRV-03, MKT-01 through MKT-06.
3. **Transaction core:** TXN-01 through TXN-04.
4. **Money:** PAY-01 through PAY-06.
5. **Trust:** TRU-01 through TRU-04.
6. **Communication:** COM-01 through COM-04.
7. **Operations:** ADM-01 through ADM-04.
8. **Security/privacy:** SEC-01 and SEC-02 operate throughout and must close before global freeze.
9. **Insights/delivery/policy:** ANA-01, MOB-01, MOB-02, LEG-01, DOC-01.
10. **Extension:** SOC-01 integrates with AI, audit, RBAC, marketplace, cases, analytics, and external adapters without bypassing their authority.
11. **Release:** REL-01 begins final promotion only after all required module gates permit it.

26. Responsibility and change control

26.1 Operational responsibility model

Concern	Primary owner	Required partners
Product scope and activation decisions	Product Owner	Engineering, operations, legal, finance, security
Runtime/application behavior	Engineering	QA, platform, security
Schema/migration/data integrity	Database/engineering owner	Platform, QA, finance/privacy where affected
Marketplace and user operations	Admin operations	Support, provider operations
Money and reconciliation	Finance Admin/finance owner	Engineering, gateway operator, audit
KYC/listing/privacy compliance	Compliance/privacy owner	Legal, security, engineering
Detection and incident response	SOC	Platform, engineering, privacy/legal as applicable
Deployment and recovery	Platform/release owner	Engineering, database, SOC, product Owner
Documentation and evidence	Module owner	QA/reviewer and affected operators

26.2 Controlled change procedure

1. Identify the exact requirement and authorized decision maker.
2. Identify affected module IDs, frozen scopes, actors, states, APIs, data, integrations, controls, tests, and manuals.
3. Record privacy, legal, financial, security, and operational impact.
4. Implement only within the approved scope.
5. Apply required migrations/data changes through controlled paths.
6. Run focused and regression acceptance.
7. Update evidence, registers, runbooks, and this documentation.
8. Close and freeze a new version only after review; never overwrite the meaning of an older accepted baseline.

27. Traceability summary

Documentation section	Governing source
Sections 1-3	Master Module Register; Master Plan §§1-2 and 23-24; status registry
Section 4	Register FND-01-FND-05; Master Plan Group A and Audit Group O
Section 5	Register IDN-01-IDN-05; Master Plan Group B
Section 6	Register PRV-01-PRV-03; Master Plan Group C
Section 7	Register MKT-01-MKT-06; Master Plan Group D
Section 8	Register TXN-01-TXN-04; Master Plan Group E
Section 9	Register PAY-01-PAY-06; Master Plan Groups F and L
Section 10	Register TRU-01-TRU-04; Master Plan Groups G, H, and J
Section 11	Register COM-01-COM-04; Master Plan Groups I and K
Section 12	Register ADM-01-ADM-04; Master Plan Group M
Section 13	Register SEC-01-SEC-02; Master Plan Groups N, O, and R
Section 14	Register ANA-01, MOB-01, MOB-02; Master Plan Groups P and Q
Section 15	Register LEG-01, DOC-01; Master Plan Groups R and S
Section 16	Register REL-01; Master Plan Group T and Phases 10-13
Section 17	Social-media architecture, workflow, RBAC, security, adapter, test, and operations evidence
Sections 18-26	Master Plan cross-module requirements and final working registries

28. Glossary

1. **Acceptance:** Evidence-backed confirmation that stated behavior and controls work in the named environment and scope.
2. **Adapter:** A provider-neutral contract that prevents external SDKs from becoming domain authority.
3. **Audit trail:** Protected event history recording actor, action, target, time, context, and outcome.
4. **Closed/Frozen:** A formally accepted exact scope protected from unrelated change; not automatically deployed.
5. **Escrow/holding:** Internal/provider accounting concept whose legal meaning must be separately confirmed.
6. **Evidence:** Reproducible code, data, test, runtime, provider, operational, or approved decision material supporting a claim.
7. **Gate:** A mandatory promotion checkpoint that cannot be inferred or skipped.
8. **GMV:** Gross merchandise value under an explicitly documented marketplace calculation.
9. **Idempotency:** Repeating the same logical request/event does not repeat its business effect.

10. **IDOR:** Insecure direct object reference; unauthorized access caused by missing object ownership/scope enforcement.
11. **KYC/KYB:** Know Your Customer/Know Your Business identity and business verification.
12. **LOCAL-RC1:** The first global local release candidate created only after all stated prerequisites pass.
13. **NO-GO:** Activation is expressly prohibited regardless of code or readiness artifacts.
14. **PSGC:** Philippine Standard Geographic Code reference data used by the address system.
15. **RBAC:** Role-based access control, supplemented by ownership, scope, state, and step-up requirements.
16. **Reconciliation:** Comparison of internal records with authoritative external/financial evidence and resolution of differences.
17. **Step-up:** Additional authentication required for a high-risk action.
18. **System of record:** The authoritative persisted source for a given state or fact.

29. Final application statement

RENTipid is a broad, evidence-oriented rental marketplace platform with substantial implementation and accepted local engineering across identity, marketplace, rental, finance, trust, administration, privacy, AI, social promotion, mobile direction, and SOC operations. Its design emphasizes authoritative state, least privilege, ownership, transaction integrity, audit, provider abstraction, reconciliation, reversible security response, and controlled release.

The truthful current posture is not “all modules are live.” The authoritative register records a mixture of not-started, in-implementation, locally accepted, and exact frozen scopes. External services and production state require separate evidence and authorization. This distinction is part of the application specification and must be preserved in product, engineering, operations, sales, training, and release communications.